Install Red Hat Device Edge on NVIDIA Jetson Orin and IGX Orin

Learning path | 3 resources | Published on September 12, 2024

Install Red Hat Device Edge on an NVIDIA® Jetson Orin™/NVIDIA IGX Orin™ Developer Kit and explore new features brought by rpm-ostree.

Path resource: Validate the Jetson GPU support

Overview: Install Red Hat Device Edge on NVIDIA Jetson Orin and IGX Orin

1 **JetPack preparation and physical cable layout**
Page

2 **Validate the Jetson GPU support**
Page

Page

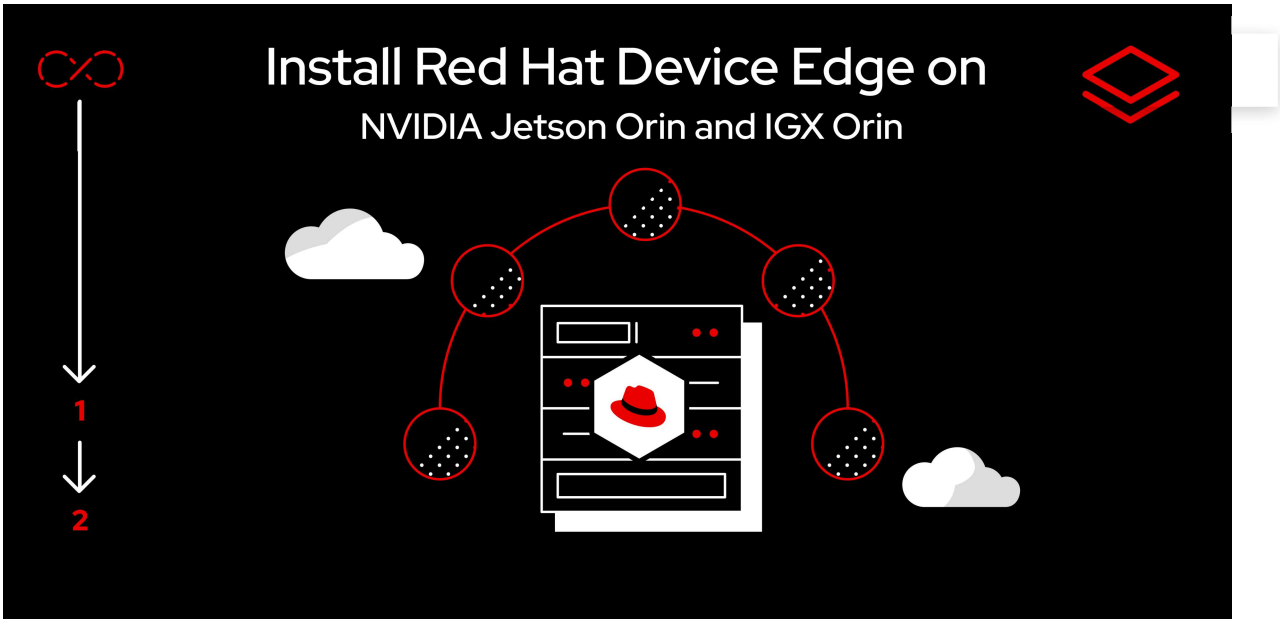
Validate the Jetson GPU support

September 12, 2024

Ben Cohen,
Henry Geay de Montenon

Related topics: Disconnected environments, Edge computing

Related products: Red Hat Enterprise Linux



This lesson walks you through the steps to run sample applications of the JetPack SDK components.

In order to get full benefit from taking this lesson, you need to:

- Set up RHEL on a Jetson device (Lesson 1).

In this lesson, you will:

- Run sample NVIDIA containers and applications to validate the GPU support.

L4T JetPack container

The L4T JetPack container contains all JetPack SDK components like CUDA, cuDNN, TensorRT, VPI, and Jetson Multimedia. Please refer to

the L4T JetPack container page on the NVIDIA GPU Cloud (NGC) for more details.

Let's run sample applications for each of these components now. Follow the steps below to proceed.

Pull the L4T JetPack container from the NGC:

```
$ podman pull nvcr.io/nvidia/l4t-jetpack:r36.3.0
```

Copy snippet

Note: The size of this container is ~14GB. Check available storage on your device before pulling the container image. For the rootless containers, the Podman uses the `${HOME}/.local/share/containers/storage` directory for storing container images (`rhel-home` partition).

CUDA

Use the following steps to run the CUDA samples in the L4T JetPack container:

1. Get CUDA 12 samples:

```
$ sudo dnf install -y git
$ cd ${HOME}
$ git clone -b v12.2 https://github.com/NVIDIA/cuda-samples.git
```

Copy snippet

2. Log in to the RHEL desktop and check the display TTY using the command w and set up the `DISPLAY` environment variable based on the command output:

```
$ export DISPLAY=:0
$ xhost +local:
```

Copy snippet

3. Run the L4T JetPack container:

```
$ podman run --rm -it \
 -e DISPLAY --net=host \
 --device nvidia.com/gpu=all \
 --group-add keep-groups \
 --security-opt label=disable \
 -v ${HOME}/cuda-samples:/cuda-samples \
 nvcr.io/nvidia/l4t-jetpack:r36.3.0
```

Copy snippet

4. Build the CUDA 12 samples in the L4T JetPack container:

```
# apt update
# apt install -y libglfw3 libglfw3-dev libdrm-dev pkg-config cmake

# cd /cuda-samples
# make clean
# make -j$(nproc)
```

Copy snippet

5. Run some CUDA samples in the L4T JetPack container:

```
# ./bin/aarch64/linux/release/deviceQuery
# ./bin/aarch64/linux/release/particles
# ./bin/aarch64/linux/release/nbody -fullscreen
```

Copy snippet

cuDNN

Use the following steps to run the cuDNN samples in the L4T JetPack container:

1. Run the L4T JetPack container:

```
$ podman run --rm -it \
 -e DISPLAY --net=host \
 --device nvidia.com/gpu=all \
```

```
 --group-add keep-groups \
 --security-opt label=disable \
 -v ${HOME}/cuda-samples:/cuda-samples \
 nvcr.io/nvidia/l4t-jetpack:r36.3.0
```

Copy snippet

2. Build and run the cuDNN samples in the L4T JetPack container:

```
# cd /usr/src/cudnn_samples_v8/conv_sample/
# make -j$(nproc)
# ./conv_sample
```

Copy snippet

TensorRT

Use the following steps to run the TensorRT samples in the L4T JetPack container:

1. Run the L4T JetPack container:

```
$ podman run --rm -it \
 -e DISPLAY --net=host \
 --device nvidia.com/gpu=all \
 --group-add keep-groups \
 --security-opt label=disable \
 -v ${HOME}/cuda-samples:/cuda-samples \
 nvcr.io/nvidia/l4t-jetpack:r36.3.0
```

Copy snippet

2. Build and run the TensorRT samples in the L4T JetPack container:

```
# cd /usr/src/tensorrt/samples/
# make -j$(nproc)
# cd ..
# ./bin/sample_algorithm_selector
# ./bin/sample_onnx_mnist
# ./bin/sample_onnx_mnist --useDLACore=0
# ./bin/sample_onnx_mnist --useDLACore=1
```

Copy snippet

DeepStream Triton container

The DeepStream Triton container can be used to run GStreamer, DeepStream, and Triton applications. Please refer to the NGC DeepStream container on the NVIDIA GPU Cloud (NGC) for more details.

Pull the DeepStream Triton container from the NGC:

```
$ podman pull nvcr.io/nvidia/deepstream:7.0-triton-multiarch
```

Copy snippet

Note: The size of this container is ~14GB. Check available storage on your device before pulling the container image. For the rootless containers, the Podman uses the `${HOME}/.local/share/containers/storage` directory for storing container images (`rhel-home` partition).

GStreamer and DeepStream

Use the following steps to run the GStreamer and DeepStream samples in the DeepStream container:

1. Log in to the RHEL desktop and check the display TTY using the command w and set up the DISPLAY environment variable based on the command output:

```
$ export DISPLAY=:0
$ xhost +local:
```

Copy snippet

2. To achieve the best performance, set the max clock settings:

```
$ sudo jetson_clocks
```

Copy snippet

3. Run the DeepStream Triton container (see the NGC DeepStream container page for more details):

```
$ podman run --rm -it \
 --env DISPLAY --net=host \
 --env=GST_PLUGIN_PATH=/usr/lib64/gstreamer-1.0 \
 --device nvidia.com/gpu=all \
 --group-add keep-groups \
 --security-opt label=disable \
 nvcr.io/nvidia/deepstream:7.0-triton-multiarch
```

Copy snippet

4. Install/run prerequisites:

```
# /opt/nvidia/deepstream/deepstream/user_additional_install.sh
```

Copy snippet

5. Run the video decode without display:

```
# gst-launch-1.0 filesrc location=/opt/nvidia/deepstream/deepstream/samples/
```

Copy snippet

6. Run video decode with output rendered to the display with both nv3d and egl sinks:

```
# gst-launch-1.0 filesrc location=/opt/nvidia/deepstream/deepstream/samples/

# gst-launch-1.0 filesrc location=/opt/nvidia/deepstream/deepstream/samples/
```

Copy snippet

7. Run video transcode and then playback transcoded file:

```
# gst-launch-1.0 filesrc location=/opt/nvidia/deepstream/deepstream/samples/

# gst-launch-1.0 filesrc location=${HOME}/sample_720p.h265 ! h265parse ! nv
```

Copy snippet

8. Run jpeg decode:

```
# gst-launch-1.0 -v filesrc location=/opt/nvidia/deepstream/deepstream/sampl

# gst-launch-1.0 -v filesrc location=/opt/nvidia/deepstream/deepstream/sampl
```

Copy snippet

9. Run jpeg encode:

```
# gst-launch-1.0 videotestsrc num-buffers=1 ! 'video/x-raw, format=(string)I

# gst-launch-1.0 -v filesrc location=${HOME}/frame.jpg ! jpegparse ! nvv4l2
```

Copy snippet

10. Run video decode and TensorRT object detection with output rendered to the display:

```
# gst-launch-1.0 filesrc location=/opt/nvidia/deepstream/deepstream/samples/

# gst-launch-1.0 filesrc location=/opt/nvidia/deepstream/deepstream/samples/
```

Copy snippet

11. Run the DeepStream test apps:

```
# cd /opt/nvidia/deepstream/deepstream/sources/apps/sample_apps/deepstream-i

# deepstream-image-decode-app /opt/nvidia/deepstream/deepstream/samples/stre

# cd /opt/nvidia/deepstream/deepstream/sources/apps/sample_apps/deepstream-t
```

```
# deepstream-test1-app ./dstest1_config.yml

# cd /opt/nvidia/deepstream/deepstream/sources/apps/sample_apps/deepstream-t

# deepstream-test2-app ./dstest2_config.yml

# cd /opt/nvidia/deepstream/deepstream/sources/apps/sample_apps/deepstream-t

# deepstream-test3-app ./dstest3_config.yml
```

Copy snippet

12. Run the secondary gstreamer inference engine (SGIE) with output rendered to the display:

```
#### To avoid dropping frames during playback
# sed -i 's/sync=1/sync=0/' /opt/nvidia/deepstream/deepstream/samples/config

# deepstream-app -c /opt/nvidia/deepstream/deepstream/samples/configs/deepst
```

Copy snippet

13. Run 30 streams video decode and TensorRT object detection with output rendered to the display. Note that left-clicking on a video stream zooms in to that stream, and right-clicking zooms back out again:

```
#### To avoid dropping frames during playback
# sed -i 's/sync=1/sync=0/' /opt/nvidia/deepstream/deepstream/samples/config

# deepstream-app -c /opt/nvidia/deepstream/deepstream/samples/configs/deepst
#### Observe the perf rate, reported in fsp.
**PERF:  15.90 (15.73) …
```

Copy snippet

GStreamer and DeepStream with USB webcam

Use the following steps to run the USB webcam examples in the DeepStream container:

1. Connect a USB webcam.
2. Re-generate the CDI spec using the NVIDIA Container Toolkit:

```
$ sudo nvidia-ctk cdi generate --mode=csv --output=/etc/cdi/nvidia.yaml
```

Copy snippet

3. Run the DeepStream Triton container:

```
$ podman run --rm -it \
 --env DISPLAY --net=host \
 --env=GST_PLUGIN_PATH=/usr/lib64/gstreamer-1.0 \
 --device nvidia.com/gpu=all \
 --group-add keep-groups \
 --security-opt label=disable \
 nvcr.io/nvidia/deepstream:7.0-triton-multiarch
```

Copy snippet

4. Install/run prerequisites:

```
# /opt/nvidia/deepstream/deepstream/user_additional_install.sh
```

Copy snippet

5. Use `v4l2-ctl` to identify the video device:

```
# apt update && apt install v4l-utils
# v4l2-ctl --list-devices
```

Copy snippet

6. Capture video and encode from a USB webcam. Set the `device="/dev/video0"` accordingly, based upon the output from `v4l2-ctl --list-devices`:

```
# gst-launch-1.0 v4l2src device="/dev/video0" num-buffers=300 ! 'video/x-ra

# gst-launch-1.0 filesrc location=${HOME}/test.mp4 !  qtdemux ! h264parse !

# gst-launch-1.0 v4l2src device="/dev/video0" num-buffers=300 ! 'video/x-ra
```

```
# gst-launch-1.0 filesrc location=${HOME}/mjpeg.avi ! avidemux ! jpegparse
```

Copy snippet

7. Capture video from a USB webcam and display. Set the `device="/dev/video0"` accordingly, based on the output from `v4l2-ctl --list-devices`:

```
# gst-launch-1.0 v4l2src device="/dev/video0" num-buffers=300 ! 'video/x-ra
```

Copy snippet

8. Capture video from a USB webcam and run through TRT:

```
# gst-launch-1.0 v4l2src device="/dev/video0" num-buffers=300 ! 'video/x-ra
```

Copy snippet

Triton

Use the following steps to run the Triton samples in the DeepStream container:

1. Set up Triton:

```
# apt update && apt -y install ffmpeg
# apt install --reinstall libavcodec58 libavutil56 libvpx7 libx264-163 libx
# cd /opt/nvidia/deepstream/deepstream/samples
# ./prepare_ds_triton_model_repo.sh
# ./prepare_classification_test_video.sh
# ./triton_backend_setup.sh
```

Copy snippet

2. Remove GStreamer cache and verify the `nvinterserver` is present:

```
# rm ~/.cache/gstreamer-1.0/registry.aarch64.bin
# gst-inspect-1.0 nvinferserver
```

Copy snippet

3. Run the sample DeepStream Triton application:

```
# cd /opt/nvidia/deepstream/deepstream/samples/configs/deepstream-app-triton
# deepstream-app -c source4_1080p_dec_infer-resnet_tracker_sgie_tiled_displa
```

Copy snippet

PyTorch

Use the following steps to run the PyTorch container:

1. Pull the PyTorch (iGPU) container from the NGC:

```
$ podman pull nvcr.io/nvidia/pytorch:24.04-py3-igpu
```

Copy snippet

Note: The size of this container is ~14GB. Check available storage on your device before pulling the container image. For the rootless containers, the Podman uses the `${HOME}/.local/share/containers/storage` directory for storing container images (`rhel-home` partition).

2. Run the PyTorch (iGPU) container:

```
$ podman run --rm -it \
 -e DISPLAY --net=host \
 --device nvidia.com/gpu=all \
 --group-add keep-groups \
 --security-opt label=disable \
 nvcr.io/nvidia/pytorch:24.04-py3-igpu
```

Copy snippet

3. Run command `deviceQuery` to detect the GPU:

```
$ deviceQuery
```

Copy snippet

4. Run the World Language Modeling example (Instructions).

Refer to the PyTorch examples and tutorials for more details.

TensorFlow

Use the following steps to run the TensorFlow container:

Pull the TensorFlow (iGPU) container from the NGC:

```
$ podman pull nvcr.io/nvidia/tensorflow:24.04-tf2-py3-igpu
```

Copy snippet

Note: The size of this container is ~10GB. Check available storage on your device before pulling the container image. For the rootless containers, the Podman uses the `${HOME}/.local/share/containers/storage` directory for storing container images (`rhel-home` partition).
Run the TensorFlow (iGPU) container:

```
$ podman run --rm -it \
       -e DISPLAY --net=host --ipc=host \
 --device nvidia.com/gpu=all \
 --group-add keep-groups \
 --security-opt label=disable \
 nvcr.io/nvidia/tensorflow:24.04-tf2-py3-igpu
```

Copy snippet

Run the command `deviceQuery` to detect the GPU:

```
$ deviceQuery
```

Copy snippet

Refer to the TensorFlow tutorials and user guide for more details.

Congratulations! You've successfully installed Red Hat Device Edge on your NVIDIA® Jetson Orin™/NVIDIA IGX Orin™ device and run several sample applications from the JetPack SDK components. Refer to the tables below for additional information.

Supported features

Feature	Subset	Supported
Hardware	IGX (iGPU)	Yes
	IGX (dGPU)	No
	AGX Orin	Yes
	Orin Nano	Yes
	Orin NX	Yes
RHEL Installation Destination	NVMe	Yes
	eMMC	Yes
	SD Card	Yes

	USB Flash Drive	Not tested.
Display & Graphics	X11	Yes
	Wayland	No
Connectivity	Ethernet	Yes
	WIFI	Yes
	USB	Yes
	PCIe (Root port)	Yes
	PCIe (Endpoint)	No
	I2C	Yes
	GPIOs	Yes[0]
	UART	Yes
	SPI	No

	CAN	Yes[0]
	PWM	Yes[0]
	HDMI (Input)	No
System, Power and Thermal	RTC	Yes
	Watchdog	Yes[0]
	Thermal sensors & fan control	Yes[0]
	Nvpmodel & Jetson Clocks	Yes
Cameras	CSI Camera	No
	USB Camera	Yes
Audio	HDMI/DP audio playback	No
	I2S audio playback/capture	No
CUDA	Native	No

	Container	Yes
TensorRT	Native	No
	Container	Yes
Multimedia	Multimedia Encode / Decode (Container)	Yes[0]
	Build and Run Multimedia API Samples	Not tested
GStreamer	Native	No
	Container	Yes
Deepstream	Native	No
	Container	Yes
Triton	Native	No
	Container	Yes
VPI	Native	No

Container	Yes[0]
Safety Extension Package (IGX)	No
CX7 (IGX)	No

[0]: This feature is available in the Tech Preview with limited functionality due to known Issues.

Known issues

The following general system usability-related issues are noted in this release.

Issue	Description
RHEL-32439	The `plymouthd` service consumes 100% CPU when no monitor is connected. This issue can be avoided by adding `plymouth.enable=0` to the kernel command line to disable the `plymouthd` service.
RHEL-36698	Installing RHEL 9.4 using the Graphical User Interface (GUI) does not work on the NVIDIA IGX Orin platform. To use the GUI installer, the user must edit the RHEL9.4 installer's kernel cmdline via the GRUB boot menu and append `'modprobe.blacklist=ast'` to the kernel cmdline.

4589352	Some NVMe cards are not detected after a warm reset and the device may require a cold reset to detect the NVMe card again. If this issue occurs, we recommend that the user check with the NVMe card manufacturer to see if there is a firmware update available for the card and if so update the firmware. If there is no firmware update available or the firmware update does not resolve the issue, then we recommend using a different NVMe card. Users can verify if their NVMe card has this problem by triggering a kernel crash, using the following command, and see if the NVMe card is detected after the device reboots: `$ echo c | sudo tee /proc/sysrq-trigger`

RHEL-32687	Jetson AGX Orin ethernet interface sometimes fails to obtain an IP address using DHCP. If the ethernet interface fails to obtain an IP address, the user can try rebooting or reloading the ethernet driver using the following commands:
	
	`$ sudo rmmod dwmac_tegra`
	
	`$ sudo modprobe dwmac_tegra`
	
	Alternatively, the user can use WiFi or connect a USB-ethernet adapter to the target for networking connectivity.

4221414	When booting RHEL9.4 on Jetson and IGX platforms the following kernel warnings are observed: `alg: self-tests for tegra-se-hmac-sha384 (hmac(sha384)) failed (rc=-75)`

	`alg: self-tests for tegra-se-hmac-sha512 (hmac(sha512)) failed (rc=-75)` These warnings are expected and don't cause any issues.
4699357	Bluetooth does not work on the Jetson AGX Orin, Orin NX, and Orin Nano devices with RHEL 9.4.
4674166	VPI sample that uses PVA does not work from the l4t-jetpack container (Harris Corners Detector sample).
RHEL-41209	No crashdump file is generated after kernel crash on Orin NX and Orin Nano.
RHEL-35045	Console output overwrites GDM display shortly after boot after installing the RHEL 9.4. Issue not is observed after installing the JetPack drivers and userspace components.
4704110	Attaching a supported MIPI (CSI) camera to an Orin NX crashes the kernel. Note: the CSI camera interface is not supported in the Tech Preview.
RHEL-43457	Intermittent kernel panic on multiple iterations of cold boot cycles on AGX Orin.
4650009	Due to an IGX device-tree bug, PCIe devices on PCIe slot 5 are not available (dGPU, CX7, and audio controller). Note: dGPU, CX7, and audio are not supported features for the Tech Preview.

The following fix can be applied to the IGX device tree before flashing the QSPI firmware:

```
# Update the IGX device-tree
```

```
cd ${HOME}/nvidia-jetson/Linux_for_Tegra/
```

```
fdtput -t x kernel/dtb/tegra234-p3740-0002+p3701-0008-nv.dtb /bus@0/pcie@141a0000 ranges 81000000 0 3a100000 0 3a100000 0 100000 82000000 0 40000000 2e 30000000 0 8000000 c3000000 28 0 28 0 6 20000000
```

```
# Reflash the QSPI firmware to IGX
```

```
sudo ./flash.sh p3740-0002-p3701-0008-qspi external
```

How to submit a bug report

If you encounter an apparent bug, you can submit a bug report.

1. Prepare a description of the bug and the procedure for reproducing it.
2. Create an sosreport and open a Red Hat support case

Submit the completed bug report in the NVIDIA Jetson or IGX support forum and attach any applicable log files.

Congratulations! Interested in learning more? Try this learning path.

← **Previous resource**
JetPack preparation and physical cable layout

Platforms

Build

Quicklinks

Communicate

RED HAT DEVELOPER

Build here. Go anywhere.

We serve the builders. The problem solvers who create careers with code.

Join us if you're a developer, software engineer, web designer, front-end designer, UX designer, computer scientist, architect, tester, product manager, project manager or team lead.

Sign me up →

About Red Hat
Jobs
Events
Locations
Contact Red Hat
Red Hat Blog
Inclusion at Red Hat
Cool Stuff Store
Red Hat Summit

© 2026 Red Hat

Privacy statement

Terms of use

All policies and guidelines

Digital accessibility

Cookie Preferences and Opt-Out Rights